Zoo Animal Tracker

```
// 1️⃣ Abstract Class
abstract class Animal(val name: String) {
    abstract fun makeSound()
}

// 3️⃣ Interface
interface Feedable {
    fun feed(food: String)
}

// 2️⃣ Subclasses

class Lion(name: String) : Animal(name), Feedable {

    override fun makeSound() {
        println("$name: Roar!")
    }

    fun hunt() {
        println("$name is hunting in the jungle.")
    }

    override fun feed(food: String) {
        println("$name eats $food.")
    }
}

class Elephant(name: String) : Animal(name), Feedable {
```

```
    override fun makeSound() {
        println("$name: Trumpet!")
    }

    fun sprayWater() {
        println("$name sprays water from its trunk.")
    }

    override fun feed(food: String) {
        println("$name enjoys eating $food.")
    }
}

class Parrot(name: String) : Animal(name) {

    override fun makeSound() {
        println("$name: Chirp!")
    }

    fun fly() {
        println("$name is flying around the zoo.")
    }
}

// 4️⃣ displayZoo Function
fun displayZoo(animals: List<Animal>) {
    println("\n--- Zoo Tracker ---")
    for (animal in animals) {
        animal.makeSound()
    }
}
```

```
// 5️⃣ Main Function
fun main() {

    val lion = Lion("Lion")
    val elephant = Elephant("Elephant")
    val parrot = Parrot("Parrot")

    val zoo: List<Animal> = listOf(lion, elephant, parrot)

    displayZoo(zoo)

    // Feeding animals (Polymorphism with interface)
    lion.feed("meat")
    elephant.feed("grass")

    // Calling extra behaviors
    lion.hunt()
    elephant.sprayWater()
    parrot.fly()
}
```